IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

Mathematical Modeling of Large Language Model Maintenance: A Stochastic Optimal-Control Framework for Retraining Scheduling, Knowledge Retention, and Alignment Preservation

Ook Lee[1]

[3]Department of Information Systems, Hanyang University, Seoul, South Korea

Corresponding author: Ook Lee (e-mail: ooklee@hanyang.ac.kr).

ABSTRACT Large language models are deployed as long-lived software artifacts whose usefulness depends on continued alignment with an evolving world, yet they are still largely treated as static objects produced once and consumed indefinitely. This paper develops a mathematical account of language-model maintenance, the recurring decision of when, how, and how much to update a deployed model, and casts it as a constrained stochastic optimal-control problem. We formalize three coupled phenomena: the loss of factual and behavioral freshness as the world drifts away from the training distribution, the risk of catastrophic forgetting whenever parameters are updated to restore freshness, and the economic cost of the update, which scales with model and data size according to empirical scaling laws. Freshness decay is modeled as a doubly stochastic renewal process, forgetting as a Fisher-information-weighted trust-region constraint on parameter displacement, and the retraining decision as a Markov decision process whose optimal policy is shown, under monotone hazard conditions, to be a control-limit (threshold) policy. We further show that retrieval-augmented generation can be interpreted as a feedback compensator that decouples the freshness objective from the parameter-update decision. The framework unifies results from the continual-learning, concept-drift, and machine-learning operations literatures in a single renewal-reward objective and yields closed-form and semi-closed-form maintenance policies. Numerical illustrations show how the optimal retraining threshold, the blend of parametric and retrieval-based updates, and the long-run maintenance cost respond to drift intensity, forgetting sensitivity, and model scale.

INDEX TERMS Catastrophic forgetting, concept drift, continual learning, large language models, Markov decision processes, model maintenance, renewal theory, retraining scheduling, retrieval-augmented generation, scaling laws.

I. INTRODUCTION

A. MOTIVATION

A large language model is, at the moment its training run ends, already beginning to become obsolete. The corpus on which it was trained is a snapshot of a world that continues to change: new facts are established, old facts are superseded, vocabulary and norms shift, and the population of tasks users bring to the model evolves. At the same time, the model's own behavior—the outputs it produces under reinforcement learning from human feedback (RLHF) or other post-training procedures—is itself a target that organizations wish to keep aligned with changing safety, policy, and product requirements. The practical response to this obsolescence is *maintenance*: some combination of full retraining, continual fine-tuning, targeted knowledge editing, and retrieval augmentation, applied at some cadence, to keep the deployed model acceptably close to a moving target.

Maintenance of statistical models is not a new problem; it is the subject matter of the concept-drift and MLOps literatures for classical machine learning models [1], [2], [3]. What is new with LLMs is the combination of (a) extremely high update costs that scale with model and data size in a manner now well characterized by empirical scaling laws [4], [5]; (b) an update mechanism—gradient-based fine-tuning of an enormous, highly entangled parameter vector—that is well known to induce catastrophic forgetting of previously

acquired competence [6], [7], [8]; and (c) a growing menu of update modalities other than full retraining, including parameter-efficient fine-tuning, targeted knowledge editing [9], [10], and retrieval-augmented generation (RAG) [11], each with a different cost/forgetting/freshness profile. Practitioners must therefore make a compound decision—*whether* to update, *how much* to update, and *by which mechanism* to update—repeatedly, under uncertainty about how quickly the world (and the acceptable behavior of the model) is drifting.

Despite the practical urgency of this decision problem, it is typically addressed heuristically: fixed retraining calendars, ad hoc drift alarms, or qualitative judgment calls about how much fine-tuning is "too much." The continual-learning community has produced a rich set of algorithms for mitigating forgetting during an update [6], [8], [12], and the concept-drift community has produced a rich set of triggers for deciding when a classical model should be retrained [1], [2], [13], [3]. What is comparatively underdeveloped is a *unifying mathematical account* that treats the decision of whether/how/how-much to maintain an LLM as a single constrained optimization problem, in which freshness decay, forgetting risk, and cost are all first-class, jointly optimized quantities rather than separately tuned heuristics.

B. PROBLEM STATEMENT

This paper addresses the following question: *Can the maintenance of a deployed large language model be formulated as a well-posed mathematical optimization problem, and if so, what is the structure of its optimal solution?* We decompose this question into three sub-problems.

1) **Freshness modeling.** How should the decay of a deployed model's task-relevant accuracy be represented mathematically as a function of time and of the (unobserved) rate at which the world changes?
2) **Forgetting-constrained update modeling.** Given a decision to update the model, how should the trade-off between absorbing new information and retaining old competence be formalized, and what does this imply about the feasible set of update magnitudes?
3) **Scheduling under cost.** Given models for freshness decay and forgetting, what is the cost-minimizing (or utility-maximizing) long-run policy for deciding when to invoke which maintenance mechanism, and what qualitative shape does that policy take?

C. CONTRIBUTIONS

This paper makes four contributions:

- It proposes a doubly stochastic renewal-process model of knowledge staleness that generalizes the deterministic decay curves implicit in empirical retraining-cadence studies [14] to a setting with random drift-event arrivals and random drift magnitudes, in the spirit of the concept-drift literature [1].
- It reformulates elastic weight consolidation (EWC) [6] and related Fisher-information regularization schemes as an explicit constraint on the feasible parameter-displacement set during an update, which allows the "amount of forgetting purchased" by an update to be expressed as a scalar that enters the maintenance cost function directly.
- It derives, via the renewal-reward theorem and a discrete-time Markov decision process (MDP) formulation, sufficient conditions under which the optimal maintenance policy is a control-limit (threshold) policy in the staleness state, connecting the LLM maintenance problem to classical reliability and replacement theory [15] and to recent results on resource allocation under concept drift with monotone hazard structure [13].
- It formalizes retrieval-augmented generation as a feedback compensator acting on the observed staleness signal without displacing model parameters, and characterizes analytically the conditions under which blending retrieval and parametric updates dominates either mechanism alone.

D. PAPER ORGANIZATION

Section II reviews the relevant literature across four strands: continual learning and catastrophic forgetting in LLMs, knowledge editing and retrieval augmentation, concept-drift-triggered retraining in classical MLOps, and the scaling laws and alignment literature that determine the cost side of the problem. Section III gives a formal definition of the LLM maintenance problem, introducing notation for staleness, forgetting, and cost that is used throughout the remainder of the paper. Section IV develops the stochastic renewal model of knowledge decay and derives the renewal-reward-optimal fixed-threshold retraining policy. Section V develops the Fisher-information-constrained model of update-induced forgetting and characterizes the stability–plasticity Pareto frontier. Section VI integrates both components into a Markov decision process and proves the threshold structure of the optimal maintenance policy under monotone drift-hazard assumptions. Section VII provides a numerical illustration of the framework's qualitative predictions. Section VIII discusses retrieval augmentation as a compensating controller. Section IX discusses limitations, and Section X concludes.

II. BACKGROUND AND HISTORY

A. HISTORICAL DEVELOPMENT

The problem of deciding when to maintain a deteriorating system predates machine learning. Reliability theory formalized replacement and maintenance policies for equipment whose condition degrades over time, including age- and block-replacement policies that trade the cost of intervention against the cost of failure [15]. Renewal theory supplies the tools for analyzing such policies over an infinite horizon, most notably the renewal-reward theorem [16], and

the theory of Markov decision processes extends them to settings with several possible actions and a controlled state [17], [18]. In machine learning, the analogous problem emerged when models were deployed on data streams whose distribution changes over time, giving rise to the concept-drift literature and its retraining triggers [1]. The arrival of transformer-based language models [19] and their scaling to few-shot learners [20] changed the economics of the problem: retraining became vastly more expensive, while updates to a single, highly entangled parameter vector became more likely to erase prior competence [8], [6]. The remainder of this section reviews the four strands of work on which this paper builds.

B. CATASTROPHIC FORGETTING AND CONTINUAL LEARNING IN LLMS

Catastrophic forgetting—the tendency of a neural network trained sequentially on new data to overwrite the representations needed for earlier tasks—has been a central concern of continual learning since its formulation in the connectionist literature and remains so in the LLM era, where it acquires new manifestations tied to instruction tuning, alignment, and continual pre-training [8], [12]. Shi et al. [8] survey the space of mitigation techniques, organizing them into replay-based methods, which rehearse a subset of prior data or synthetic surrogates during an update; regularization-based methods, which penalize movement of parameters deemed important to earlier tasks; and architecture-expansion methods, which allocate new capacity for new tasks while freezing capacity associated with old ones. Wu et al. [12] propose a complementary categorization organized around the stage of the LLM lifecycle at which continual adaptation occurs—continual pre-training, continual instruction tuning, and continual alignment—and note that the sheer cost of retraining LLMs from scratch makes incremental adaptation not merely convenient but close to necessary at scale.

Empirically, Luo et al. [7] conduct a systematic study of forgetting during continual instruction tuning of LLMs ranging from one to seven billion parameters, evaluating forgetting along the axes of domain knowledge, reasoning ability, and reading comprehension. Their central and somewhat counter-intuitive finding is that forgetting *intensifies* with model scale within the range studied, which the authors attribute to larger models having more capability to lose in the first place; they also find that decoder-only architectures forget less than encoder–decoder architectures of comparable scale, and that broad instruction tuning prior to a narrower fine-tuning stage can attenuate subsequent forgetting. These findings matter for a mathematical theory of maintenance because they imply that the forgetting-penalty term in any maintenance cost function cannot be treated as scale-invariant: larger, more capable, and more heavily aligned models are not automatically more robust to further updates, and in some regimes may be less so.

The dominant mathematical device for controlling forgetting during an update is elastic weight consolidation (EWC), introduced by Kirkpatrick et al. [6]. EWC casts sequential learning as approximate Bayesian filtering: the posterior over parameters after training on an initial task is approximated, via a Laplace approximation around the maximum-a-posteriori solution, by a Gaussian whose precision matrix is the diagonal of the Fisher information matrix evaluated at that solution. When a subsequent task's loss is optimized, this approximate posterior is folded in as a quadratic penalty on the displacement of each parameter from its prior value, weighted by that parameter's estimated importance to the earlier task. Follow-up work has both refined and criticized the practical behavior of this penalty: several studies note that its regularizing effect can diminish when applied to the fine-tuning of large pre-trained language models unless its components are rescaled, and that its Fisher-information weights, computed from squared log-likelihood gradients, are themselves noisy estimates that can misjudge which parameters truly matter to prior competence. For the purposes of this paper, EWC and its relatives are important not for their algorithmic details but for what they reveal structurally: that any update to a deployed LLM can, in principle, be described as the solution to a *constrained* optimization problem, in which the constraint set is defined by an allowable radius of parameter displacement measured in a Fisher-information-weighted metric. This is the formal device adopted in Section V.

C. KNOWLEDGE EDITING AND RETRIEVAL AUGMENTATION

An alternative to full or partial retraining is to modify a narrow slice of a model's stored knowledge directly. Wang et al. [9] survey knowledge-based model editing (KME), formalizing the goal as locating and rewriting a specific fact within a pre-trained model's parameters while leaving unrelated behavior unaffected—a substantially more surgical intervention than gradient-based fine-tuning over a broad dataset. Zhang et al. [10] extend this survey with a benchmark and a systematic study of where factual knowledge is localized within transformer layers, motivated explicitly by the same economic pressure this paper takes as a starting assumption: that naively retraining an LLM every time a fact becomes outdated is computationally prohibitive, and that a lighter-weight editing mechanism is therefore economically attractive whenever the update in question is narrow in scope.

A structurally different response to knowledge staleness is retrieval-augmented generation (RAG). Gao et al. [11] survey the RAG paradigm, in which a model retrieves passages from an external, independently updatable corpus at inference time and conditions its generation on that retrieved evidence, rather than relying solely on knowledge encoded in its parameters at training time. Because the corpus can be updated continuously and cheaply relative to the cost of retraining the model itself, RAG effectively separates the *freshness* of the information a

model can draw upon from the *currency* of its parameters. This separation is central to Section VIII of this paper, where we argue that RAG is best modeled not as a substitute for the maintenance decision but as an additional control input that acts on the observed staleness signal without touching the parameter vector, and therefore without incurring forgetting risk.

D. CONCEPT DRIFT AND RETRAINING SCHEDULING IN CLASSICAL MLOPS

The question of *when* to retrain a deployed statistical model, as opposed to *how* to retrain it, has a long history in the concept-drift literature. Lu et al. [1] provide a comprehensive review of concept drift, distinguishing between sudden, gradual, incremental, and recurring drift patterns and cataloguing detection statistics used to trigger retraining. Two retraining philosophies recur throughout this literature and are directly relevant to the maintenance problem addressed here: *periodic* retraining, in which the model is refreshed on a fixed calendar regardless of observed performance, and *informed* (or triggered) retraining, in which a drift-detection statistic determines the retraining time endogenously. A companion design axis concerns which data are used at retraining time—a *sliding-window* approach that discards old data in favor of recent observations, versus a *full-history* approach that accumulates all data seen to date—each of which implies a different bias–variance and forgetting profile.

Mahadevan and Mathioudakis [2] formalize the retraining-timing decision explicitly as a cost-minimization problem over a data stream, proposing a cost-aware retraining algorithm that adapts to the joint statistics of drift severity and retraining expense, and showing empirically that a cost-aware policy dominates naive drift-triggered baselines both in accuracy and in the number of retraining events incurred. Beytur et al. [13] go further and study the underlying resource-allocation problem analytically: modeling the durations of stationary "concept" regimes as random variables and deriving optimal training policies under a fixed budget, they show that the qualitative shape of the optimal policy is governed by the *aging* property of the concept-duration distribution—whether the distribution has decreasing or increasing mean residual life—and that heuristics which ignore this aging property can be provably suboptimal. This is one of the few treatments in the literature that derives, rather than assumes, the structural form of an optimal retraining policy, and Section VI of this paper builds directly on the renewal-theoretic and optimal-control tools used there. A retrospective clinical study by Williams et al. [14] offers a complementary, purely empirical data point: in a five-year rolling simulation of an emergency-department admission model, retraining cadences ranging from weekly to triennial were compared directly, with monthly retraining found to sustain discrimination and calibration at roughly one quarter of the computational cost of weekly retraining—an observation consistent with a *diminishing-returns* relationship between retraining frequency and maintained accuracy, a relationship this paper makes explicit and general in Section IV.

E. COST STRUCTURE: SCALING LAWS AND ALIGNMENT

Any maintenance policy is a policy over costly actions, and the cost of the dominant action—retraining or fine-tuning—is itself now understood with unusual empirical precision. Kaplan et al. [4] establish that the pre-training loss of a transformer language model follows a power law in model size, dataset size, and training compute across many orders of magnitude, with architectural details such as depth and width mattering comparatively little. Hoffmann et al. [5] refine this picture by jointly optimizing model size and token count for a fixed compute budget, showing that many contemporary models had been substantially over-parameterized relative to the volume of data on which they were trained, and establishing the compute-optimal scaling relation used in Section III of this paper to translate an update's size and scope into an expected monetary or compute cost.

A further cost dimension specific to LLMs is *alignment maintenance*. Ouyang et al. [21] show that fine-tuning a pre-trained language model with human feedback—first via supervised imitation of human demonstrations and then via reinforcement learning against a learned reward model—substantially improves the model's helpfulness and truthfulness as judged by human evaluators, to the point that a much smaller aligned model can be preferred to a much larger unaligned one. Their study also documents an "alignment tax": naive reinforcement fine-tuning degrades performance on some public NLP benchmarks relative to the base model, a cost the authors mitigate by mixing the original pre-training objective into the reinforcement-learning update. This is a direct, empirically documented instance of the stability–plasticity trade-off that Section V formalizes: the alignment update is itself a maintenance action that purchases improved conformity to current human preferences at some risk of degrading previously acquired general capability, and the mitigation strategy—blending old and new objectives—is structurally identical to the Fisher-information-weighted penalty used in EWC.

F. SYNTHESIS AND GAP

Table I summarizes how the four strands of literature reviewed above map onto the components of the maintenance problem formalized in Section III. Continual learning and knowledge editing address *how* an update should be executed once a decision to update has been made; concept-drift research addresses *when* an update should be triggered for classical, comparatively cheap-to-retrain models; scaling laws and alignment research quantify *how costly* an update is once its size and scope are fixed. No single strand jointly optimizes the *when*, *how much*, and *by which mechanism* decisions under a shared cost objective that includes both forgetting risk and

monetary cost, which is precisely the integration this paper undertakes.

TABLE I
LITERATURE STRANDS MAPPED TO THE COMPONENTS OF THE MAINTENANCE PROBLEM.

Strand	Representative work	Question addressed	What it leaves open
Continual learning and forgetting	[8], [7], [12], [6]	How to execute an update without losing prior competence	When to update; monetary cost
Knowledge editing and retrieval	[9], [10], [11]	How to update narrowly, or without touching parameters	Scheduling; cost model
Concept drift and retraining scheduling	[2], [13], [1], [14], [3]	When to trigger an update	Developed for cheap-to-retrain classical models
Scaling laws and alignment	[4], [5], [21]	How costly an update is	Retention and scheduling decisions

III. FORMAL DEFINITION OF LLM MAINTENANCE

A. THE DEPLOYED MODEL AS A TIME-INDEXED OBJECT

Let $\theta_t \in \mathbb{R}^p$ denote the parameter vector of a deployed LLM at time $t \geq 0$, with θ_0 the parameters produced by the initial training run. Between maintenance events, θ_t is held fixed. A maintenance event at time t_k replaces $\theta_{t_k^-}$ with $\theta_{t_k^+} = \theta_{t_k^-} + \Delta_k$, where Δ_k is the parameter displacement induced by the k-th maintenance action; a full retraining corresponds to Δ_k that is unconstrained in norm (and typically very large), a knowledge edit or parameter-efficient fine-tune corresponds to a Δ_k that is sparse or low-rank, and a retrieval-augmentation event corresponds to $\Delta_k = 0$ together with an update to an external, independently indexed corpus $\mathcal{C}_t$.

B. FRESHNESS AND STALENESS

We define an (unobservable in general, but statistically estimable) *world state* $w_t \in \mathcal{W}$ evolving as a stochastic process representing the distribution of facts, tasks, and behavioral norms the model will be asked to handle at time t. The model's *task-relevant accuracy*, or freshness, is a functional

$$A(\theta_t, w_t) = \mathbb{E}_{x \sim w_t}\big[u(\theta_t, x)\big] \tag{1}$$

where $u(\theta, x)$ is a bounded per-example utility (for example, an indicator of correctness, a human-preference score, or a negative loss). Staleness at time t since the last maintenance event at t_k is defined as

$$S(t) = A(\theta_{t_k^+}, w_{t_k}) - A(\theta_{t_k^+}, w_t) \tag{2}$$

the accuracy the deployed model *would have retained had the world not moved*, minus its accuracy against the *current* world. $S(t)$ is non-negative in expectation whenever w_t drifts away from w_{t_k} on average, and $S(t_k) = 0$ immediately after a maintenance event. Staleness is the quantity that continual-learning, knowledge-editing, and retrieval-augmentation interventions all ultimately aim to reduce, and it plays the role in this paper that a wear or damage process plays in classical reliability theory.

C. FORGETTING

Whenever a maintenance action induces a nonzero displacement Δ_k, it risks degrading accuracy on the *portion of the world distribution already mastered*, an effect distinct from, and in the worst case additive to, staleness. We define the *forgetting* incurred by an update as

$$G(\Delta_k) = A(\theta_{t_k^-}, w_{t_k^-}) - A(\theta_{t_k^-} + \Delta_k, w_{t_k^-}) \tag{3}$$

the drop in accuracy against the *pre-update* world distribution caused solely by the parameter displacement, holding the world fixed. $G(\Delta_k) \geq 0$ is the empirically documented phenomenon studied in [8], [7], [6], and it is a function of both the magnitude and the *direction* of Δ_k relative to directions of high curvature in the pre-update loss landscape, a dependency made explicit in Section V.

D. COST

Each maintenance action carries an operational cost. Guided by the compute-optimal scaling relation of Hoffmann et al. [5], the cost of a maintenance action that touches N_{eff} effective parameters over D_k tokens of update data is modeled as

$$c(\Delta_k) = \kappa \cdot 6\, N_{\text{eff}}(\Delta_k)\, D_k \tag{4}$$

where κ converts floating-point operations to a monetary or opportunity cost and the factor of six reflects the standard forward-plus-backward-pass FLOP accounting used in scaling-law analyses [4], [5]. Full retraining sets $N_{\text{eff}}(\Delta_k) \approx p$; parameter-efficient fine-tuning or targeted knowledge editing sets $N_{\text{eff}}(\Delta_k) \ll p$; retrieval-augmentation events instead incur a much smaller, largely θ-independent indexing cost c_{RAG} that scales with corpus size rather than model size.

E. THE MAINTENANCE OBJECTIVE

Collecting the three quantities above, the long-run maintenance problem is to choose a (possibly randomized, possibly history-dependent) sequence of maintenance times $\{t_k\}$, displacements $\{\Delta_k\}$, and retrieval-corpus updates $\{\mathcal{C}_{t_k}\}$ to minimize the long-run average cost rate

$$J = \limsup_{T \to \infty} \frac{1}{T}\, \mathbb{E}\left[\int_0^T \big(\alpha\, S(t) + \beta \mathbb{1}\{t \in \{t_k\}\}\, G(\Delta_k)\big)\, dt + \sum_{k:\, t_k \leq T} c(\Delta_k)\right] \tag{5}$$

where α converts staleness into an operational loss rate (for example, lost users, compliance exposure, or downstream error propagation) and β converts a one-time forgetting event into an equivalent flow cost over the interval until the next maintenance event. Equation (5) is deliberately structured so

that its three additive terms map one-to-one onto Sections IV (the $S(t)$ dynamics), V (the $G(\Delta_k)$ dynamics), and the $c(\Delta_k)$ cost model already given above; Section VI shows how to jointly optimize over $\{t_k, \Delta_k\}$ under this objective using dynamic programming.

F. MODELING ASSUMPTIONS AND THEIR SCOPE

Three simplifying assumptions are maintained throughout the paper and flagged again in Section IX. First, $A(\theta, w)$ is treated as a scalar summary of accuracy; in practice a deployed model is evaluated along many task-specific axes that can drift at different rates, a complication briefly revisited in Section IX. Second, the world process w_t is treated as exogenous—the model's own outputs are assumed not to feed back into the distribution of tasks or facts it will later be asked about, an assumption that is increasingly strained as LLM-generated content becomes part of the training data for future models and future world states. Third, maintenance actions are assumed to be discrete events; continuous, always-on adaptation is a limiting case of the discrete framework as the minimum inter-event time shrinks to zero, and is not separately treated.

IV. A STOCHASTIC RENEWAL MODEL OF KNOWLEDGE STALENESS

A. DRIFT AS A MARKED POINT PROCESS

We model the arrival of *drift events*—discrete moments at which the world state w_t shifts enough to be behaviorally consequential (a new fact supersedes an old one, a policy changes, a new class of user request emerges)—as a marked point process $\{(\tau_i, M_i)\}_{i\geq 1}$, where τ_i are arrival times and $M_i \geq 0$ are i.i.d. drift magnitudes independent of the arrival process, drawn from a distribution F_M with mean μ_M. In the simplest case the arrival process is Poisson with rate λ, giving a compound Poisson staleness accumulation process; more generally, as emphasized by Beytur et al. [13] and consistent with the taxonomy of sudden, gradual, and incremental drift in Lu et al. [1], the inter-arrival times may follow a general distribution F_τ with hazard rate $h_\tau(\cdot)$, allowing drift "seasons" to exhibit increasing or decreasing mean residual life.

Between drift events, staleness decays the model's freshness continuously through ordinary exposure to a slowly moving world (vocabulary drift, slow factual turnover), modeled as a deterministic leak term, and jumps discretely at each drift event. The resulting staleness process obeys the stochastic differential equation

$$dS(t) = \eta\, S(t)\, dt + \sum_{i:\, \tau_i \in (t_k, t]} M_i\, \delta(t - \tau_i) \tag{6}$$

with $S(t_k^+) = 0$ immediately after the k-th maintenance event, $\eta \geq 0$ a background drift-leak rate, and the sum ranging over drift events since the last maintenance. Equivalently, in expectation,

$$\mathbb{E}[S(t)] = \eta\,(t - t_k) + \Lambda(t - t_k)\,\mu_M \tag{7}$$

where $\Lambda(u) = \mathbb{E}[N(u)]$ is the renewal function of the drift-arrival process over an interval of length u. In the Poisson special case $\Lambda(u) = \lambda u$, recovering the familiar result that expected staleness grows affinely in time since last maintenance, with slope $\eta + \lambda\mu_M$—a single "effective decay rate" that a practitioner could in principle estimate from monitoring data, exactly as advocated in drift-detection practice [1].

B. A RENEWAL-REWARD FORMULATION OF THE RETRAINING DECISION

Consider first the restricted policy class in which maintenance is always full retraining (so $G(\Delta_k) \equiv 0$ by construction, since the retrained model is refit to the current world) triggered whenever accumulated staleness first exceeds a fixed threshold θ^*. This defines a renewal process on the staleness state: each maintenance event resets $S(t)$ to zero and the system regenerates probabilistically. Let $T(\theta^*) = \inf\{u \geq 0 : \mathbb{E}[S(u)] \geq \theta^*\}$ denote the (expected) cycle length implied by threshold θ^*, and let c_r denote the retraining cost from Section III-D. By the renewal-reward theorem [16], the long-run average cost rate under this policy is

$$J(\theta^*) = \frac{c_r + \alpha \int_0^{T(\theta^*)} \mathbb{E}[S(u)]\, du}{T(\theta^*)} \tag{8}$$

Substituting the affine staleness growth $\mathbb{E}[S(u)] = \gamma u$ with $\gamma \equiv \eta + \lambda\mu_M$, and noting $T(\theta^*) = \theta^*/\gamma$, the integral evaluates in closed form to $\alpha\gamma T(\theta^*)^2/2 = \alpha(\theta^*)^2/(2\gamma)$, giving

$$J(\theta^*) = \frac{c_r \gamma}{\theta^*} + \frac{\alpha\, \theta^*}{2} \tag{9}$$

Minimizing over θ^* by setting $dJ/d\theta^* = 0$ yields the classical square-root economic-order-quantity form familiar from inventory and reliability theory,

$$\theta^{**} = \sqrt{\frac{2\, c_r\, \gamma}{\alpha}}, \qquad J(\theta^{**}) = \sqrt{2\, c_r\, \gamma\, \alpha} \tag{10}$$

Two qualitative predictions follow immediately and are consistent with the empirical retraining-cadence literature. First, the optimal staleness-tolerance threshold—and hence the implied optimal *time* between retraining events, $T^\star = \theta^{**}/\gamma = \sqrt{2c_r/(\gamma\alpha)}$—shrinks as the drift-intensity parameter γ grows, matching the qualitative finding of Williams et al. [14] that faster clinical or operational drift favors more frequent retraining. Second, $T^\star$ grows only as the square root of the retraining cost c_r, so an order-of-magnitude increase in the cost of retraining a larger model (as implied by the scaling relation of Section III-D) only lengthens the optimal retraining interval by roughly a factor of three, not by an order of magnitude—a diminishing-returns relationship consistent with the observation in [14] that monthly retraining preserved most of the benefit of weekly retraining at a quarter

of the cost, and providing a first mathematical explanation for why naive "just retrain less because it is expensive" heuristics can be badly miscalibrated.

C. BEYOND THE POISSON CASE: HAZARD-RATE-DEPENDENT POLICIES

When drift arrivals are not Poisson, the renewal function $\Lambda(u)$ is no longer linear, and the threshold policy of Section IV-B, while still well defined, is no longer necessarily optimal among *all* history-dependent policies; a time-based or hybrid trigger may dominate a pure staleness-threshold trigger. Following the aging-property analysis of Beytur et al. [13], we distinguish two regimes. If the inter-drift-event time distribution has *decreasing mean residual life* (DMRL)—meaning that the longer the system has gone without a drift event, the sooner the next one is expected—then a policy that also shortens the retraining interval as time-since-last-event grows without a drift event strictly dominates the memoryless threshold policy of Section IV-B. If instead the distribution has *increasing mean residual life* (IMRL), the reverse holds and the naive threshold policy is close to optimal, while a policy that treats long quiet periods as a signal to retrain sooner is provably suboptimal. This result is significant for practice because the two regimes are empirically distinguishable from monitoring data (via estimation of the hazard rate of drift-detector alarms) and imply *opposite* prescriptions for how a retraining scheduler should react to an unusually long quiet interval, a distinction invisible to policies that only monitor accumulated staleness and ignore elapsed time since the last observed drift event.

D. ESTIMATING THE DECAY PARAMETERS FROM MONITORING DATA

The parameters η, λ, and μ_M (or, more generally, the renewal function $\Lambda(\cdot)$) are not directly observed but can be estimated from a monitoring pipeline that periodically evaluates the deployed model against held-out, freshly collected evaluation data—the same operational infrastructure already recommended in the concept-drift literature [1] and increasingly standard in MLOps practice for classical models [2], [3]. Because $\mathbb{E}[S(u)]$ is, under the model of Section IV-A, affine in u between maintenance events, γ can be recovered by ordinary least-squares regression of measured staleness on elapsed time since the last maintenance event, using data pooled across multiple maintenance cycles; departures from affinity in this regression are themselves diagnostic of non-Poisson drift arrivals and motivate the hazard-rate-based refinement of Section IV-C.

V. FISHER-INFORMATION-CONSTRAINED UPDATE DYNAMICS: A MODEL OF CATASTROPHIC FORGETTING

A. FORGETTING AS A FUNCTION OF DISPLACEMENT GEOMETRY

Following the Bayesian derivation underlying elastic weight consolidation [6], we approximate the pre-update loss surface $\mathcal{L}(\theta_{t_k^-} + \Delta)$, restricted to the *old* world distribution $w_{t_k^-}$, by its second-order Taylor expansion around $\theta_{t_k^-}$:

$$\mathcal{L}(\theta_{t_k^-} + \Delta) \approx \mathcal{L}(\theta_{t_k^-}) + \tfrac{1}{2}\,\Delta^\top F\,\Delta \tag{11}$$

where F is the (diagonal-approximated) Fisher information matrix of the pre-update model with respect to the old task distribution, and the first-order term vanishes because $\theta_{t_k^-}$ is (approximately) a local minimum of the pre-update loss. Under this approximation, the forgetting functional defined in Section III-C is, to leading order, a quadratic form in the displacement:

$$G(\Delta) \approx \tfrac{1}{2}\,\Delta^\top F\,\Delta = \tfrac{1}{2}\sum_{i=1}^{p} F_i\,\Delta_i^2 \tag{12}$$

This is precisely the penalty term added to the training objective under EWC and its variants [6]; the contribution of this paper is to treat $G(\Delta)$ not merely as a regularizer to be added to a loss function during optimization, but as an explicit *constraint set* entering the maintenance cost objective of Section III, so that the amount of forgetting an organization is willing to "purchase" in a given update becomes a tunable, auditable quantity rather than an implicit consequence of a regularization hyperparameter.

B. THE CONSTRAINED UPDATE PROBLEM

Given a target reduction in staleness $\bar{S}$ that a maintenance action of type k is intended to achieve, and given the quadratic forgetting approximation above, the update-selection problem at maintenance time t_k is

$$\Delta_k^\star = \arg\min_{\Delta}\; c(\Delta) \quad \text{s.t.} \quad \Delta^\top F\,\Delta \le 2\gamma_{\max}, \;\; \text{(new-data loss reduction)} \ge \bar{S} \tag{13}$$

where $\gamma_{\max}$ is an organization's tolerance for forgetting expressed in the same accuracy units as staleness, and $c(\Delta)$ is the cost functional of Section III-D. Because $c(\Delta)$ is, for a fixed choice of update *mechanism* (full retraining, low-rank fine-tuning, sparse knowledge edit), approximately proportional to the number of effective parameters touched—that is, to $\|\Delta\|_0$ or a soft relaxation thereof—this problem exhibits a direct trade-off between the *sparsity/locality* of the update, its *cost*, and its *forgetting footprint*, with the Fisher information matrix F acting as an anisotropic metric that makes some directions in parameter space far more "expensive" (in forgetting terms) to move along than others. This formalizes, in closed mathematical form, the widely observed empirical fact that knowledge-editing and parameter-efficient fine-tuning methods—which restrict Δ to a low-dimensional or sparse subspace by construction [9], [10]—achieve much lower forgetting than unconstrained full-parameter fine-tuning for a comparable gain in target-task performance.

C. THE STABILITY–PLASTICITY PARETO FRONTIER

Define the *plasticity* achieved by an update as the staleness reduction it delivers, $P(\Delta) \equiv S(t_k^-) - S(t_k^+;\Delta)$, and its *stability*

cost as the forgetting $G(\Delta)$ from Section V-A. For a fixed cost budget $c(\Delta) \le \bar{c}$, tracing out the efficient frontier of the bi-objective problem $\max P(\Delta)$, $\min G(\Delta)$ over admissible Δ produces a Pareto frontier whose shape is governed by the alignment between the gradient direction that reduces staleness on the new data and the principal eigenvectors of F. When these directions are nearly orthogonal—that is, when the knowledge needed to address the new drift event occupies parameter directions to which the old task is insensitive—the frontier is nearly flat and large plasticity gains can be purchased at negligible stability cost; this is the regime knowledge-editing methods are designed to exploit by explicitly localizing new facts to under-used parameter subspaces [9], [10]. When the two directions are nearly parallel—for instance, when a broad alignment update of the kind studied by Ouyang et al. [21] necessarily moves parameters that are heavily used across almost the entire pre-training distribution—the frontier is steep, and every unit of new-behavior plasticity is bought at substantial cost to old-behavior stability, which is exactly the "alignment tax" documented empirically in [21] and mitigated there by blending the reinforcement-learning update with the original pre-training loss, a strategy that is formally a projection of Δ onto a subspace with reduced average F-weighted norm.

D. INCORPORATING FORGETTING INTO THE RENEWAL-REWARD OBJECTIVE

Because a nonzero forgetting event $G(\Delta_k)$ effectively creates an *additional* staleness-like liability at the moment of maintenance—the model is now further from optimal on the old distribution, in exactly the units used to measure staleness against the new distribution—Section IV's renewal-reward analysis extends naturally by replacing the zero post-update staleness assumption $S(t_k^+) = 0$ with $S(t_k^+) = G(\Delta_k) \ge 0$. Repeating the derivation of Section IV-B with this modification and a forgetting-cost coefficient β yields a long-run cost rate

$$J(\theta^*, \Delta) = \frac{c(\Delta) + \beta\, G(\Delta) + \alpha\left(\frac{(\theta^* - G(\Delta))^2}{2\gamma} + G(\Delta)\, T(\theta^*)\right)}{T(\theta^*)} \tag{14}$$

whose minimization over the *joint* choice of threshold θ^* and update mechanism Δ (equivalently, over the point chosen on the Pareto frontier of Section V-C) is precisely the integration promised in Section III-E: a mechanism that achieves lower forgetting at a given cost shifts the effective post-update staleness baseline downward, which in turn justifies a *larger* optimal threshold θ^* and hence a *longer* optimal interval between maintenance events, all else equal—a quantitative version of the intuitive claim that safer update mechanisms (knowledge editing, low-rank adaptation, retrieval augmentation) should be preferred not only because they forget less per event but because they change the economically optimal *cadence* of maintenance itself.

VI. A MARKOV DECISION PROCESS FORMULATION OF MAINTENANCE SCHEDULING

A. STATE, ACTION, AND TRANSITION STRUCTURE

To move beyond the single-threshold policies of Sections IV and V and allow for a menu of maintenance mechanisms with distinct cost/forgetting/staleness-reduction profiles, we discretize time into decision epochs $t = 0, 1, 2, \dots$ and define the state at each epoch as $x_t = (S_t, d_t) \in \mathcal{S} \times \mathcal{D}$, where S_t is the (discretized) staleness level from Section IV and d_t is an estimate of the current drift-hazard regime from Section IV-C (for instance, a coarse indicator of whether the elapsed-time-since-last-drift-event process is currently in a DMRL- or IMRL-consistent phase, updated via a filtering procedure applied to drift-detector alarms). At each epoch the maintainer chooses an action $a_t \in \mathcal{A} = \{\text{no-op}, \text{RAG-refresh}, \text{knowledge-edit}, \text{parameter-efficient fine-tune}, \text{full retrain}\}$, each associated with a cost $c(a)$ (Section III-D), a forgetting increment $G(a)$ (Section V), and a staleness-reduction fraction $\rho(a) \in [0, 1]$ such that $S_{t+1} = (1 - \rho(a_t))S_t + G(a_t) + \xi_{t+1}$, where $\xi_{t+1} \ge 0$ is the (random) staleness increment accrued during the epoch under the drift model of Section IV, with mean and variance determined by d_t.

B. THE BELLMAN EQUATION

The per-epoch cost is $r(x_t, a_t) = \alpha S_t + \beta G(a_t) + c(a_t)$, and the long-run average-cost optimality equation (the Bellman equation for this MDP, following standard dynamic-programming theory [18], [17]) is

$$h(x) + g = \min_{a \in \mathcal{A}} \left\{ r(x, a) + \mathbb{E}\left[h(x') \mid x, a\right] \right\} \tag{15}$$

where g is the (unknown, to-be-solved-for) optimal long-run average cost rate and $h(\cdot)$ is a relative value function unique up to an additive constant. Because $\mathcal{A}$ is a small, finite set and $\mathcal{S} \times \mathcal{D}$ can be discretized to any desired resolution, Equation (15) is directly solvable by standard relative value iteration or policy iteration for any specific numerical instantiation of the cost, forgetting, and drift parameters, and does not require closed-form solution in general; the purpose of this section is instead to establish *qualitative structural properties* of the optimal policy that hold across a broad range of parameterizations.

C. MONOTONICITY AND THE THRESHOLD-POLICY THEOREM

Proposition 1 (Threshold structure). Suppose that (i) for each fixed drift regime d, the transition kernel $S_{t+1} \mid S_t, a$ is stochastically non-decreasing in S_t (a higher current staleness level implies a higher, in the stochastic-dominance sense, staleness level next period, for any fixed action); (ii) the action set $\mathcal{A}$ is totally ordered by increasing $\rho(a)$ and increasing $c(a)$ (more aggressive maintenance actions are both more effective and more expensive, the empirically natural ordering no-op $\prec$ RAG-refresh $\prec$ knowledge-edit $\prec$ parameter-efficient fine-tune $\prec$ full retrain); and (iii) $r(x, a)$ is

submodular in (S, a) (the marginal benefit of a more aggressive action is non-decreasing in current staleness). Then the relative value function $h(S, d)$ is non-decreasing and convex in S for each fixed d, and the optimal policy $a^\star(S, d)$ is monotone non-decreasing in S: there exist thresholds $0 \le S_1(d) \le S_2(d) \le \cdots \le S_{|\mathcal{A}|-1}(d)$ such that the optimal action escalates through the ordered action set as S crosses each threshold in turn.

Sketch. The result follows from the standard monotone-MDP argument (see, e.g., the structural-results tradition surveyed in the dynamic-programming and reliability-replacement literatures): submodularity of the one-step cost together with a stochastically monotone transition kernel is preserved under the Bellman operator, so if $h(\cdot, d)$ is convex and non-decreasing at one iterate of value iteration, the next iterate inherits the property; the fixed point $h(\cdot, d)$ of Equation (15) therefore also has the property, and monotone comparative statics on the minimizer of a submodular function in a monotone variable then yields a monotone (here, threshold-structured) minimizer. ■

This proposition generalizes the single-threshold result of Section IV-B (recovered as the special case $|\mathcal{A}| = 2$, i.e., a choice only between no-op and full retrain, with $F \to 0$ so forgetting plays no role) to the realistic case of a graduated menu of maintenance mechanisms, and it formally justifies practices already emerging informally in MLOps pipelines—escalating from a light-touch retrieval-corpus refresh, to a targeted knowledge edit, to a broader parameter-efficient fine-tune, and only as a last resort to full retraining—as the *provably optimal* qualitative shape of a maintenance policy, rather than a merely convenient engineering heuristic, under the monotonicity conditions stated. The dependence of the thresholds $S_j(d)$ on the drift regime d further formalizes the DMRL/IMRL distinction of Section IV-C [13]: in a regime d associated with imminent, high-hazard drift, the thresholds shift downward, causing the policy to escalate to more aggressive maintenance actions at lower observed staleness than it would in a low-hazard regime, exactly the qualitative behavior derived by Beytur et al. [13] for the two-action (train-or-not) case and generalized here to a graduated action set.

D. NUMERICAL SOLUTION VIA RELATIVE VALUE ITERATION

For any specific parameterization of the cost, forgetting, and drift-transition functions introduced above, Equation (15) can be solved numerically without appeal to Proposition 1 at all, which provides both a practical route to deployment and an independent check on the proposition's predictions. Relative value iteration proceeds by fixing a reference state x_{ref}, initializing $h_0(x) \equiv 0$, and iterating

$$h_{n+1}(x) = \min_{a \in \mathcal{A}} \{r(x, a) + \mathbb{E}[h_n(x') \mid x, a]\} - \min_{a \in \mathcal{A}} \{r(x_{\mathrm{ref}}, a) + \mathbb{E}[h_n(x') \mid x_{\mathrm{ref}}, a]\} \tag{16}$$

until h_{n+1} and h_n agree to within a specified tolerance, at which point the minimizing action at each state is read off as the (numerically) optimal policy and the subtracted term converges to the optimal average cost rate $g^\star$. Because the state space $\mathcal{S} \times \mathcal{D}$ used in this paper is low-dimensional by design (a scalar discretized staleness level crossed with a small number of drift-hazard regimes), this computation is inexpensive relative to the cost of even a single knowledge-editing operation on a deployed model, and can in principle be re-solved on a rolling basis as the underlying drift parameters are re-estimated from monitoring data as described in Section IV-D, yielding an adaptive rather than purely static maintenance policy. When the monotonicity conditions of Proposition 1 hold, the numerically obtained policy is guaranteed to exhibit the threshold structure derived analytically; when they do not, relative value iteration remains valid and simply returns whatever (possibly non-monotone) policy is in fact optimal, making it a useful diagnostic for detecting violations of the proposition's assumptions in a fitted, real-world instantiation of the model.

E. DISCUSSION OF THE STRUCTURAL RESULT

Two caveats qualify Proposition 1's practical reach. First, condition (i)—stochastic monotonicity of the staleness transition in current staleness—can fail if a maintenance action's forgetting increment $G(a)$ interacts with staleness non-monotonically (for instance, if attempting an aggressive fine-tune from an already very stale starting point induces disproportionately severe forgetting because the optimizer must travel further in parameter space); Section V's Pareto-frontier analysis suggests this is a real possibility whenever the required displacement direction rotates substantially as staleness accumulates. Second, condition (iii)'s submodularity assumption encodes a specific, economically plausible but not universally guaranteed belief about how the *relative* attractiveness of aggressive versus conservative actions changes with staleness; violations would produce a genuinely non-monotone optimal policy, which is a testable prediction of the model rather than an assumption to be taken on faith.

VII. NUMERICAL RESULTS AND FINDINGS

A. PURPOSE AND SETUP

This section illustrates the qualitative behavior implied by Sections IV–VI using stylized parameter values chosen for interpretability rather than calibration to any specific deployed system; the purpose is to make the comparative statics of the model concrete, not to produce a validated forecast. We instantiate the single-threshold model of Section IV-B with a background leak rate $\eta = 0.01$ per day, a drift-event rate λ swept over $\{0.05, 0.1, 0.2, 0.4\}$ events per day, a mean drift magnitude $\mu_M = 1$ (in staleness units normalized so that a single "typical" drift event, if left unaddressed for one day, would move accuracy by one unit), and an operational-loss coefficient $\alpha = 1$. Retraining cost c_r is swept over three orders of magnitude, $\{10^2, 10^4, 10^6\}$, to represent a small fine-tune, a mid-size continual-pretraining run, and a large-scale full retrain respectively, loosely consistent with the compute-

cost scaling of Section III-D applied to models differing by roughly two orders of magnitude in parameter count.

B. QUALITATIVE PREDICTIONS

Under the closed-form solution of Section IV-B, $\theta^{**} = \sqrt{2c_r\gamma/\alpha}$ with $\gamma = \eta + \lambda\mu_M$, three qualitative patterns emerge and are worth stating explicitly as testable implications of the model. First, holding c_r fixed, the optimal retraining interval $T^\star = \sqrt{2c_r/(\gamma\alpha)}$ falls by a factor of two whenever the drift intensity γ quadruples, so a fourfold acceleration in the rate at which the world changes around a deployed model justifies only a *halving*, not a quartering, of the maintenance interval—a much more conservative adjustment than a naive "match the drift rate" heuristic would suggest, and one that follows directly from the square-root form of the economically optimal threshold. Second, holding drift intensity fixed, moving from the smallest to the largest retraining-cost regime considered here (a factor of 10^4) lengthens the optimal interval by only a factor of 100, again reflecting the diminishing sensitivity of the optimum to cost that is structurally guaranteed by the square-root relationship, and offering a mathematical explanation for the empirical observation in [14] that a large reduction in retraining frequency (from weekly to monthly, a factor of roughly four) cost comparatively little accuracy relative to the computational savings achieved. Third, incorporating the forgetting-augmented objective of Section V-D shows that for any fixed target staleness reduction, mechanisms achieving a lower forgetting increment $G(\Delta)$ per unit of cost shift the effective long-run cost curve $J(\theta^*, \Delta)$ downward and its minimizing threshold rightward simultaneously, meaning that investment in *less-forgetting* maintenance mechanisms (better knowledge localization, better regularization) pays a double dividend: it lowers the per-event cost of maintenance and it lengthens the economically optimal interval between maintenance events.

C. SENSITIVITY TO THE DRIFT-HAZARD REGIME

Instantiating the two-regime extension of Section IV-C with a Weibull inter-drift-event distribution—shape parameter greater than one corresponding to the IMRL case, shape parameter less than one corresponding to the DMRL case—reproduces the qualitative reversal identified by Beytur et al. [13]: under the DMRL parameterization, a policy that shortens the effective retraining interval as time-since-last-drift-event grows achieves a materially lower long-run cost rate than the memoryless threshold policy of Section IV-B, whereas under the IMRL parameterization the two policies are nearly indistinguishable in cost, and a policy that reacts to a long quiet spell by retraining sooner performs strictly worse. This confirms, in the illustrative setting, that the *shape* of the drift-arrival hazard function is not a second-order detail but a first-order determinant of which class of maintenance policy an organization should adopt, and that this determination is in principle estimable from historical drift-detector alarm data as outlined in Section IV-D.

D. SUMMARY OF FINDINGS

The analytical and numerical results of Sections IV through VIII can be summarized as five findings. First, under Poisson drift arrivals the optimal interval between retraining events grows with the square root of the retraining cost and shrinks with the square root of the drift intensity (Section IV-B). Second, whether a memoryless threshold policy is adequate depends on the aging property of the drift-arrival distribution: under decreasing mean residual life an elapsed-time-sensitive policy dominates, whereas under increasing mean residual life the threshold policy is close to optimal (Sections IV-C and VII-C). Third, forgetting can be expressed as a Fisher-weighted quadratic form in the parameter displacement, so the stability–plasticity trade-off becomes a Pareto frontier whose steepness depends on the alignment between the update direction and the high-curvature directions of the pre-update loss (Section V). Fourth, under monotonicity and submodularity conditions, the optimal policy over a graduated menu of maintenance actions escalates through a sequence of staleness thresholds (Proposition 1). Fifth, retrieval augmentation lengthens the optimal interval between parametric updates in proportion to the share of drift it absorbs, but it cannot substitute for parametric maintenance when the drift is not retrievable (Section VIII). These findings are derived from the model and illustrated with stylized parameters; they have not yet been validated against deployment data (Section IX-E).

VIII. RETRIEVAL AUGMENTATION AS A FEEDBACK COMPENSATOR

A. A CONTROL-THEORETIC READING OF RAG

The maintenance mechanisms considered in Sections V and VI—knowledge editing, parameter-efficient fine-tuning, full retraining—all act by displacing θ_t, and therefore all incur some forgetting risk $G(\Delta_k) \geq 0$ under the model of Section V. Retrieval-augmented generation [11] is structurally different: it reduces observed staleness by conditioning generation at inference time on passages drawn from an external corpus $\mathcal{C}_t$ that can be updated independently and continuously, without touching θ_t at all. In the language of Section III-B, a well-functioning RAG system effectively replaces the freshness functional $A(\theta_t, w_t)$ with an augmented functional $A(\theta_t, \mathcal{C}_t, w_t) \geq A(\theta_t, w_t)$ whenever $\mathcal{C}_t$ contains information relevant to closing the gap between w_{t_k} and w_t, while leaving θ_t, and hence $G(\cdot)$, entirely unaffected.

This is naturally read as feedback control: if we regard staleness $S(t)$ as an error signal, a parametric update is analogous to *adjusting the plant itself* (changing θ, with attendant risk of destabilizing previously tuned behavior), whereas a retrieval-corpus refresh is analogous to *adjusting a feed-forward or compensating input* that corrects the observed

error without modifying the plant's internal dynamics at all. Formally, define an effective staleness under retrieval augmentation

$$\tilde{S}(t) = S(t) - \phi(\mathcal{C}_t, w_t) \tag{17}$$

where $\phi(\cdot) \geq 0$ is the staleness reduction attributable to retrieval, increasing in the recency and coverage of $\mathcal{C}_t$ relative to w_t and bounded above by the *retrievability* of the relevant knowledge—that is, by how well the kind of drift in question (a new fact, easily indexed) versus a change in required reasoning style or task format, which retrieval cannot easily supply, is amenable to correction via retrieved text at all.

B. OPTIMAL BLENDING OF RETRIEVAL AND PARAMETRIC MAINTENANCE

Substituting $S(t)$ for $S(t)$ throughout the renewal-reward and MDP analyses of Sections IV and VI shows immediately that maintaining an up-to-date retrieval corpus at low marginal cost $c_{\mathrm{RAG}} \ll c(\Delta)$ shifts the effective drift-leak parameter downward (since a fraction of what would otherwise register as staleness is absorbed by $\phi(\cdot)$ before it accumulates against the parametric model), which by the closed-form result of Section IV-B *lengthens* the optimal interval between parametric maintenance events, exactly as intuition would suggest, but now with an explicit, falsifiable functional form: the optimal parametric-retraining threshold under retrieval augmentation is $\theta^{\star\star}_{\mathrm{RAG}} = \sqrt{2c_r(\gamma - \bar{\phi})/\alpha}$ for an average retrieval-absorbed drift rate $\bar{\phi} < \gamma$, strictly larger than the no-RAG optimum whenever retrieval absorbs a nonzero share of drift. This formalizes the widely held but rarely quantified engineering intuition that a well-maintained retrieval system is a substitute—over the portion of drift it can absorb—for more frequent and more forgetting-prone parametric updates, while making explicit that the substitution is only partial and bounded by $\bar{\phi}$: forms of drift that are not amenable to retrieval-based correction (shifts in required reasoning patterns, output format conventions, or safety norms that are not simply "facts" to be looked up) still accumulate at their full rate against the parametric model and must eventually be addressed via the mechanisms of Sections V and VI.

C. FAILURE MODES NOT CAPTURED BY STALENESS REDUCTION ALONE

It is worth noting explicitly that $\phi(\cdot)$ can be negative in effect even when nominally positive in design if the retrieved evidence is irrelevant or contradictory, a failure mode connected in the broader literature to the general problem of hallucination in generated text [22]; the reduction in *measured* staleness attributable to RAG is therefore best treated, for maintenance-planning purposes, as an empirically monitored quantity subject to the same estimation procedure outlined in Section IV-D, rather than as a fixed design parameter that can be assumed correct once a retrieval system has been deployed.

IX. LIMITATIONS AND FUTURE WORK

A. IDENTIFIABILITY AND DATA REQUIREMENTS

A further practical limitation, distinct from the modeling simplifications discussed below, concerns identifiability: the estimation procedure of Section IV-D requires repeated observation of full maintenance cycles—staleness accumulating from a reset point, a maintenance event, and a subsequent reset—in order to fit even the simplest affine decay model, and organizations that maintain their models very infrequently, or that have changed maintenance mechanism or monitoring methodology between cycles, may simply lack the historical data needed to estimate γ, let alone the richer hazard-rate structure of Section IV-C, with any precision. In such data-poor settings, the qualitative comparative statics derived analytically in Sections IV through VI—for instance, the square-root sensitivity of the optimal interval to cost and to drift intensity—are arguably of more immediate practical value than any point estimate of the optimal threshold itself, since the former are robust, structural predictions while the latter is only as reliable as the (possibly very limited) historical record used to calibrate it.

B. SCALAR ACCURACY AND MULTI-TASK HETEROGENEITY

The framework developed here treats task-relevant accuracy $A(\theta, w)$ as a single scalar, but a deployed LLM is evaluated by its users along many largely independent axes—factual accuracy, coding ability, safety compliance, stylistic preference—each of which may drift at a different rate and respond differently to a given maintenance mechanism. A vector-valued extension of Sections III through VI, in which staleness and forgetting are tracked per task-cluster and the maintenance action space is allowed to target clusters selectively (as targeted knowledge editing already does implicitly [9], [10]), is a natural next step and would allow the threshold-policy result of Section VI to be tested for robustness under a genuinely multi-objective cost structure.

C. ENDOGENEITY OF THE WORLD PROCESS

Section III assumes the world process w_t evolves independently of the deployed model's own outputs. This assumption is already strained by the growing presence of LLM-generated text in the corpora used to evaluate and retrain subsequent models, which introduces a feedback loop between a model's maintenance history and the very drift process it is trying to track; formalizing this feedback—for instance, by allowing the drift-event rate λ in Section IV to depend on cumulative prior deployment—is left for future work and would connect this paper's renewal-theoretic framework to the broader literature on distribution shift induced by model-generated data.

D. PARAMETER ESTIMATION UNDER NON-STATIONARITY

The estimation procedure sketched in Section IV-D assumes that the parameters η, λ, μ_M governing drift are themselves stable across the observation window used to estimate them; if

these parameters are themselves slowly time-varying (a plausible scenario, since the rate at which the world changes is not itself constant), a more sophisticated online, possibly Bayesian, estimation scheme would be required, and the fixed-threshold and even the regime-dependent policies of Sections IV through VI would need to be re-derived as adaptive-control rather than fixed-parameter optimal-control problems.

E. EMPIRICAL VALIDATION

This paper is deliberately structured as a mathematical modeling contribution rather than an empirical one: its numerical illustrations in Section VII are chosen for interpretability, not fitted to observed deployment data. A natural and important next step is to instantiate the model—the staleness-decay parameters of Section IV, the Fisher-information-based forgetting model of Section V, and the action-cost structure of Section VI—against telemetry from an actual production LLM deployment, and to test the central falsifiable predictions of the framework: the square-root relationship between retraining cost and optimal interval (Section IV-B), the diminishing sensitivity of the optimum to drift intensity (Section VII-B), the threshold-escalation structure of the optimal multi-action policy (Proposition 1), and the partial-substitution relationship between retrieval augmentation and parametric maintenance frequency (Section VIII-B).

X. CONCLUSION

This paper has argued that the maintenance of a deployed large language model—the recurring decision of when, how much, and by which mechanism to update it—can be given a coherent mathematical treatment that unifies previously separate strands of the continual-learning, concept-drift, and MLOps literatures under a single renewal-reward optimization objective. Modeling knowledge staleness as a doubly stochastic renewal process, modeling update-induced forgetting as a Fisher-information-weighted constraint on parameter displacement in the tradition of elastic weight consolidation [6], and modeling the joint scheduling and mechanism-selection decision as a Markov decision process yields both closed-form results in restricted cases—most notably a square-root relationship between retraining cost, drift intensity, and the economically optimal maintenance interval—and a general structural theorem establishing that, under economically natural monotonicity conditions, the optimal policy over a graduated menu of maintenance mechanisms takes the form of an escalating sequence of staleness thresholds. The framework further shows that retrieval-augmented generation is properly understood not as an alternative to parametric maintenance but as a feedback compensator that partially and boundedly substitutes for it, absorbing the share of drift that is amenable to retrieval while leaving the residual, non-retrievable component of drift to be addressed through the forgetting-constrained parametric mechanisms characterized in Section V. Beyond its specific formulas, the central claim of this paper is methodological: that treating LLM maintenance as a first-class, jointly optimized stochastic control problem—rather than as a collection of separately tuned heuristics for drift detection, forgetting mitigation, and cost control—both explains qualitative patterns already visible in the empirical literature and yields new, falsifiable predictions that can guide the design of maintenance policies for future deployed systems.

REFERENCES

[1] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.
[2] A. Mahadevan and M. Mathioudakis, "Cost-effective retraining of machine learning models," arXiv:2310.04216, 2023.
[3] L. Poenaru-Olaru, J. Sallou, L. Cruz, J. S. Rellermeyer, and A. van Deursen, "Sustainable machine learning retraining: Optimizing energy efficiency without compromising accuracy," arXiv:2506.13838, 2025.
[4] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," arXiv:2001.08361, 2020.
[5] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre, "Training compute-optimal large language models," arXiv:2203.15556, 2022.
[6] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proc. Natl. Acad. Sci. U.S.A.*, vol. 114, no. 13, pp. 3521–3526, 2017.
[7] Y. Luo, Z. Yang, F. Meng, Y. Li, J. Zhou, and Y. Zhang, "An empirical study of catastrophic forgetting in large language models during continual fine-tuning," arXiv:2308.08747, 2023.
[8] H. Shi, Z. Xu, H. Wang, W. Qin, W. Wang, Y. Wang, Z. Wang, S. Ebrahimi, and H. Wang, "Continual learning of large language models: A comprehensive survey," arXiv:2404.16789, 2024.
[9] S. Wang, Y. Zhu, H. Liu, Z. Zheng, C. Chen, and J. Li, "Knowledge editing for large language models: A survey," *ACM Comput. Surv.*, vol. 57, no. 3, Nov. 2024.
[10] N. Zhang, Y. Yao, B. Tian, P. Wang, S. Deng, M. Wang, Z. Xi, S. Mao, J. Zhang, Y. Ni, S. Cheng, Z. Xu, X. Xu, J.-C. Gu, Y. Jiang, P. Xie, F. Huang, L. Liang, Z. Zhang, X. Zhu, J. Zhou, and H. Chen, "A comprehensive study of knowledge editing for large language models," arXiv:2401.01286, 2024.
[11] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, Q. Guo, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.
[12] T. Wu, L. Luo, Y.-F. Li, S. Pan, T.-T. Vu, and G. Haffari, "Continual learning for large language models: A survey," arXiv:2402.01364, 2024.
[13] H. B. Beytur, G. de Veciana, H. Vikalo, and K. S. Chan, "Optimal resource allocation for ML model training and deployment under concept drift," arXiv:2512.12816, 2025.
[14] E. Williams, T. Sinha, M. Lyttle, and Y. Kanagasingam, "Optimising retraining frequency for a paediatric emergency department admission prediction model: Development and temporal validation using real-world data," *Emerg. Med. Australas.*, vol. 38, no. 3, 2026.
[15] R. E. Barlow and F. Proschan, *Mathematical Theory of Reliability*. New York, NY, USA: Wiley, 1965.
[16] S. M. Ross, *Stochastic Processes*, 2nd ed. New York, NY, USA: Wiley, 1996.
[17] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York, NY, USA: Wiley, 1994.
[18] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, 4th ed., vol. I. Belmont, MA, USA: Athena Scientific, 2017.

[19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.
[20] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020.
[21] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022.
[22] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.



OOK LEE received the Ph.D. degree in management of information systems from Claremont Graduate University, Claremont, CA, USA. He is currently a Professor with the Department of Information Systems, Hanyang University, Seoul, South Korea. He was previously a faculty member with the University of Queensland, Brisbane, QLD, Australia; the University of Nevada at Las Vegas, Las Vegas, NV, USA; and North Carolina A&T State University, Greensboro, NC, USA. His research interests include information systems economics, information systems maintenance, large language models, and critical social theory.